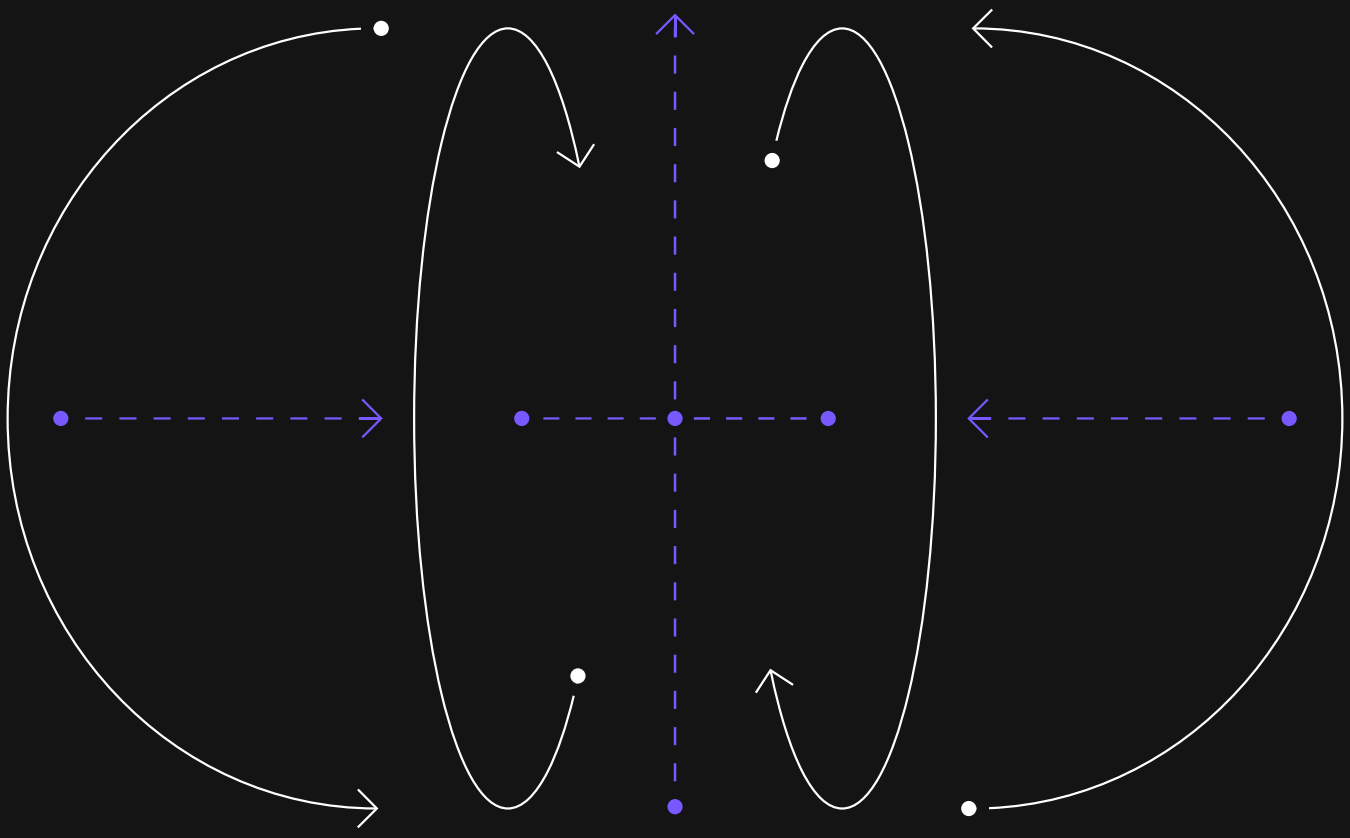
Машинное обучение | Лонгрид

Глубокое обучение | 07

Рекуррентные нейронные сети



Содержание

1 Постановки задач для последовательностей **4**

1.1 Последовательности переменной длины . . . 4

1.2 Типология задач . . . 4

2 Базовая рекуррентная сеть **5**

2.1 Вход и скрытые состояния . . . 5

3 Градиент по времени: затухание и взрыв **6**

3.1 Градиент по рекуррентной матрице . . . 6

3.2 Раскрутка по времени . . . 6

3.3 Условия затухания и взрыва градиента . . . 7

4 Стабилизация градиента **7**

4.1 Прямой путь в рекуррентном переходе . . . 8

4.2 Ортогональная рекуррентная матрица . . . 8

4.3 Обрезание градиента . . . 8

5 LSTM **9**

5.1 Состояние памяти и скрытое состояние . . . 9

5.2 Формулы ячейки . . . 9

5.3 Градиент по состоянию памяти . . . 9

5.4 Инициализация гейтов . . . 10

5.5 Вычислительная стоимость . . . 10

6 GRU **10**

7 Сборка полной сети **10**

7.1 Двунаправленные сети . . . 11

7.2 Многослойные сети и понижение временного разрешения . . . 11

7.3 Остаточные связи по глубине . . . 12

7.4 Архитектуры под типы задач . . . 12

8 Dropout в рекуррентных сетях **13**

8.1 Маски dropout 13
8.2 Вариационный dropout 14
8.3 Dropout внутри ячейки 14

9 Нормализация в рекуррентных сетях 14

10 Обучение авторегрессионного декодировщика 15

11 Анализ состояний обученной LSTM 16

12 Ограничение фиксированного контекстного вектора 16

13 Итоги 17

A Дополнительные методы стабилизации и регуляризации 17

A.1 Ортогональная параметризация через матричную экспоненту 17
A.2 Zoneout 17

Рекуррентные нейронные сети

Постановка задачи

Дана последовательность токенов $x = (x_1, \dots, x_T)$, где $x_t \in \mathcal{V}$. Эмбеддинг каждого токена $e_t = e(x_t) \in \mathbb{R}^{d_e}$ строится по материалу недели 6. Требуется получить скрытые состояния $h_t \in \mathbb{R}^{d_h}$ и по ним предсказать метку всей последовательности, метку каждого токена или выходную последовательность другой длины.

Параметры рекуррентного перехода общие для всех позиций. Для задачи с целевыми ответами y обучение минимизирует функцию потерь $L(y, p)$, например кросс-энтропию по классам или сумму кросс-энтропий по позициям, где p обозначает предсказанные распределения классов. Ниже рассматриваются базовая рекуррентная сеть, обратное распространение по времени, LSTM, GRU и кодировщик-декодировщик.

1. Постановки задач для последовательностей

1.1. Последовательности переменной длины

Полносвязный слой принимает вектор заданной размерности. Сверточный слой применяет одно и то же локальное преобразование и может работать с разными пространственными размерами, хотя батчи изображений обычно приводят к согласованной форме. Теперь нас интересует другая структура входа: упорядоченная последовательность, длина которой меняется от примера к примеру. Рекуррентная сеть использует одну и ту же ячейку для каждого ее элемента.

Текст и звук имеют переменную длину, причем менять ее могут и вход, и выход. Предложение состоит из произвольного числа токенов, аудиосигнал имеет произвольную продолжительность. Ниже рассмотрим основные сочетания форм входа и выхода.

1.2. Типология задач

- **Последовательность → объект фиксированного размера.** Классификация текстов: на входе рецензия на фильм, на выходе метка класса - положительная, отрицательная или нейтральная.
- **Объект фиксированного размера → последовательность.** Подпись к картинке: на входе фото, на выходе текстовое описание того, что на нем изображено.
- **Последовательности согласованной длины.** Разметка текста по частям речи: для каждого токена нужна своя метка класса. Это аналог сегментации изображений для текста: выходная последовательность имеет ту же длину, что и входная. Если исходная разметка задана для слов, ее предварительно согласуют с токенами выбранного токенизатора.
- **Последовательность → последовательность произвольных длин** (seq2seq). Машинный перевод: текст на одном языке - его перевод на другом, вообще говоря другой длины. Распознавание речи: звук на входе, произнесенный текст на выходе. Синтез речи - наоборот.

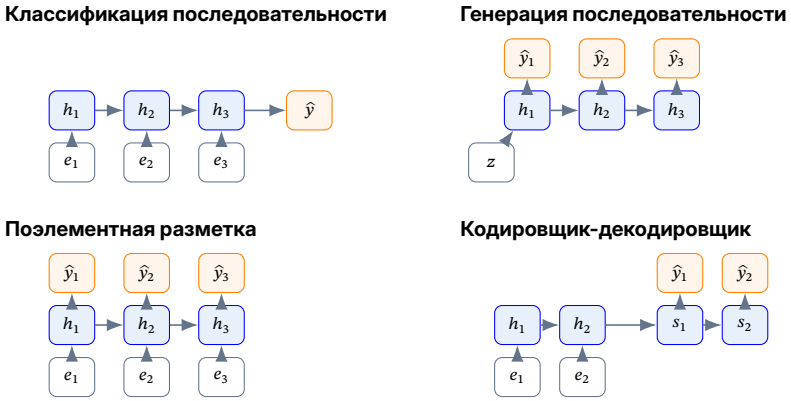


Рис. 1: Основные постановки задач для последовательностей: классификация, генерация, поэлементная разметка и преобразование одной последовательности в другую

Во всех четырех постановках одна и та же рекуррентная ячейка применяется к последовательности произвольной длины. Функция потерь зависит от формы ответа: одна кросс-энтропия для классификации последовательности или сумма потерь по позициям для разметки и генерации.

2. Базовая рекуррентная сеть

2.1. Вход и скрытые состояния

Пусть $x_t \in \mathcal{V}$ - токен в позиции t, а $e_t = e(x_t) \in \mathbb{R}^{d_e}$ - его эмбеддинг. Размер словаря равен $V = |\mathcal{V}|$, длина последовательности - T. Матрица эмбеддингов $E \in \mathbb{R}^{V \times d_e}$ и соглашение $e_t = e(x_t)$ введены на неделе 6.

Статические и контекстные представления

Статический эмбеддинг токена не зависит от позиции и контекста, а усреднение эмбеддингов стирает порядок токенов. Скрытое состояние рекуррентной сети h_t зависит от префикса $(x_1, \dots, x_t)$ и потому задает контекстное представление позиции t.

Для каждой позиции вычисляется скрытый вектор $h_t \in \mathbb{R}^{d_h}$. Он зависит от текущего эмбеддинга и предыдущего скрытого состояния. Начальное состояние h_0 обычно задают нулевым или обучаемым вектором:

$$h_t = \varphi\big(W_e e_t + W_h h_{t-1} + b\big),$$

где $W_e \in \mathbb{R}^{d_h \times d_e}$, $W_h \in \mathbb{R}^{d_h \times d_h}$, $b \in \mathbb{R}^{d_h}$, а φ действует покомпонентно. Если ответ требуется в каждой позиции, выходной слой классификатора имеет вид

$$z_t = W_y h_t + b_y, \qquad p_t = \mathrm{softmax}(z_t), \qquad \hat{y}_t = \arg\max_{1 \le k \le K} p_{t,k}.$$

Для классификации на K классов $W_y \in \mathbb{R}^{K \times d_h}$ и $b_y \in \mathbb{R}^{K}$. Вектор z_t содержит логиты, p_t - вероятности классов, а $\hat{y}_t$ - выбранную метку. При обучении кросс-энтропия вычисляется по логитам z_t и правильной метке y_t. Все позиционные потери входят в общую функцию потерь L.

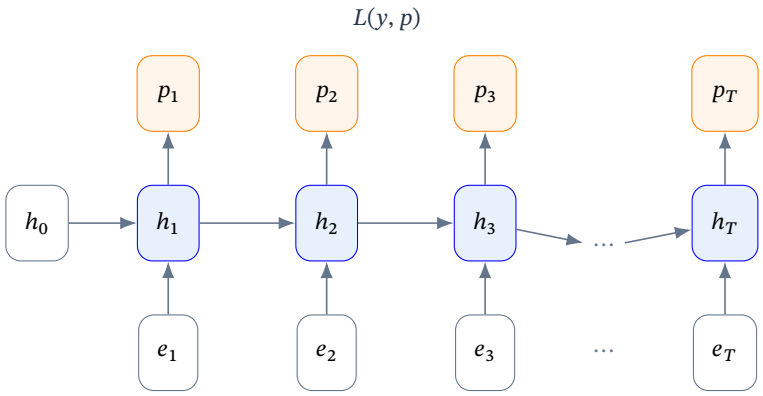


Рис. 2: Базовая рекуррентная сеть в развернутом по времени виде: вертикальные связи - вход и выход текущего момента, горизонтальные - передача скрытого состояния

Ключевое свойство: матрицы W_e, W_h, W_y и сдвиги b, b_y общие для всех позиций. Поэтому число параметров не зависит от длины последовательности, а одна и та же ячейка применяется на каждом шаге.

Это похоже на сверточный слой: одна рекуррентная ячейка применяется ко всем позициям последовательности так же, как одно сверточное ядро применяется ко всем пространственным позициям изображения.

3. Градиент по времени: затухание и взрыв

3.1. Градиент по рекуррентной матрице

Рассчитаем градиент функции потерь по рекуррентной матрице W_h. В вычислительном графе одна и та же матрица используется при вычислении каждого скрытого состояния.

Обозначим аргумент функции активации через $a_t = W_e e_t + W_h h_{t-1} + b$, так что $h_t = \varphi(a_t)$. Выпишем дифференциал:

$$dh_t = D_t\, da_t, \qquad D_t = \operatorname{diag}(\varphi'(a_t))$$

- функция активации действует покомпонентно, поэтому ее якобиан диагонален. Рассмотрим зависимость от W_h. В момент времени t соответствующее слагаемое дифференциала равно $da_t = (dW_h)\, h_{t-1}$. Сворачивая его с градиентом по h_t и используя представление скалярного произведения через след, получаем вклад момента t:

$$\langle \nabla_{h_t} L,\ D_t\,(dW_h)\,h_{t-1}\rangle = \operatorname{tr}\big(h_{t-1}\,\nabla_{h_t} L^{\mathsf{T}} D_t\, dW_h\big) \implies D_t\,\nabla_{h_t} L\, h_{t-1}^{\mathsf{T}}$$

- это матрица того же размера, что и W_h. Поскольку W_h используется на всех шагах, полный градиент равен сумме вкладов по моментам времени:

$$\nabla_{W_h} L = \sum_{t=1}^{T} D_t\,\nabla_{h_t} L\, h_{t-1}^{\mathsf{T}}.$$

3.2. Раскрутка по времени

Остался важный момент: в слагаемом $\nabla_{h_t} L$ матрица W_h тоже используется - градиент по текущему h_t выражается через градиент по следующему моменту времени. Через якобиан перехода $h_t \to h_{t+1}$:

$$\nabla_{h_t} L = \left(\frac{\partial h_{t+1}}{\partial h_t}\right)^{\mathsf{T}} \nabla_{h_{t+1}} L, \qquad \frac{\partial h_{t+1}}{\partial h_t} = D_{t+1} W_h.$$

Подставляя это выражение само в себя - h_{t+1} через h_{t+2} и так далее до конца последовательности, - получаем ключевую формулу. Для наглядности учитывается только вклад последнего момента времени; вклады промежуточных y_t устроены аналогично и картину не меняют:

$$\nabla_{W_h} L \;=\; \sum_{t=1}^{T} D_t \left(J_{T-1} \cdots J_t\right)^{\top} \nabla_{h_T} L \;\; h_{t-1}^{\top}.$$

Здесь $J_k \;=\; \partial h_{k+1} / \partial h_k$. Порядок матриц существенен: переход от h_t к h_T равен $J_{T-1} \cdots J_t$. При $t \;=\; T$ произведение пустое и равно единичной матрице.

3.3. Условия затухания и взрыва градиента

Оценим норму градиента сверху - норма суммы не больше суммы норм, норма произведения не больше произведения норм:

$$\left\| \nabla_{W_h} L \right\| \;\le\; \sum_{t=1}^{T} \|D_t\| \left(\prod_{k=t}^{T-1} \left\| \frac{\partial h_{k+1}}{\partial h_k} \right\| \right) \left\| \nabla_{h_T} L \right\| \|h_{t-1}\|.$$

В середине стоит произведение норм якобианов по времени. Если все нормы ограничены одним числом $\rho < 1$, эта верхняя граница убывает как ρ^{T-t}, и вклад удаленного состояния может затухать. Если нормы больше единицы, возможен взрыв, но сама верхняя оценка еще не доказывает его: направление градиента и взаимное действие матриц тоже имеют значение.

T обозначает длину последовательности. Для текста это могут быть сотни токенов, а две секунды звука при частоте дискретизации 44 кГц содержат 88 тысяч отсчетов. Поэтому даже небольшое систематическое уменьшение нормы при каждом переходе дает экспоненциальное затухание вклада удаленных состояний.

Глубина, пропорциональная длине

Особенность рекуррентных сетей именно в том, что глубина вычислительного графа становится пропорциональной длине последовательности. Затухание градиента было и в полносвязных сетях - уже к пятому слою градиент может ощутимо затухнуть, - и в сверточных, но там долгое время работали с единицами и десятками слоев, и эффект был куда менее выражен. Здесь глубина может составлять сотни и тысячи переходов, поэтому обучение зависимостей между удаленными позициями становится существенно труднее.

Функции активации в разных преобразованиях

Функции активации в разных местах схемы могут отличаться. Чтобы сеть моделировала нелинейные зависимости, нелинейность нужна в ее вычислениях. При этом отдельные преобразования могут оставаться линейными: например, перед softmax обычно стоит линейный выходной слой. Требование нелинейности не относится к каждой стрелке графа.

4. Стабилизация градиента

Затухание определяется произведением якобианов по времени. Рассмотрим способы сократить путь градиента и контролировать норму рекуррентного преобразования.

4.1. Прямой путь в рекуррентном переходе

Остаточная связь добавляет к рекуррентному переходу прямое слагаемое:

$$h_{t+1} = \varphi\big(W_e e_{t+1} + W_h h_t + b\big) + h_t.$$

Тогда якобиан перехода равен $I + D_{t+1} W_h$. Прямое слагаемое дает дополнительный путь для градиента, но норма такого якобиана не обязана быть близкой к единице: поправка может усиливать или подавлять некоторые направления.

4.2. Ортогональная рекуррентная матрица

Ортогональная параметризация контролирует норму линейной части якобиана:

$$\frac{\partial h_{k+1}}{\partial h_k} = D_{k+1} W_h.$$

Ортогональная матрица W_h сохраняет евклидову норму вектора при линейном преобразовании. Но полный якобиан содержит еще D_{k+1}, производную активации. У ReLU она равна нулю на отрицательной области, а у насыщенного tanh мала. Поэтому ортогональность линейной части сама по себе не устраняет затухание полного перехода. Если производная близка к единице на нужных компонентах, такой прием помогает передавать сигнал по времени.

Один из способов задать ортогональную рекуррентную матрицу через обучаемые параметры приведен в приложении.

Ограничения прямого пути и ортогональности

Остаточная связь и ортогональная матрица помогают контролировать передачу градиента, но не дают общей гарантии отсутствия затухания. В рекуррентных моделях широко применяются и гейтовые ячейки: они позволяют обучать, какую информацию сохранять, какую обновлять и что выдавать наружу. Далее разберем LSTM и GRU.

4.3. Обрезание градиента

Обрезание градиента по норме ограничивает слишком большой шаг оптимизатора:

$$g_{\text{clip}} = \begin{cases} g, & \|g\| \le \tau, \\ \tau \dfrac{g}{\|g\|}, & \|g\| > \tau, \end{cases}$$

где τ - заданный порог. Если норма градиента превышает τ, его масштаб уменьшается при сохранении направления.

Обрезание не устраняет причину взрыва градиента, но ограничивает норму конкретного обновления и стабилизирует обучение.

Частое срабатывание обрезания градиента - повод проверить нормы градиентов, шаг оптимизации и динамику функции потерь. Порог выбирают с учетом задачи, а качество проверяют по обучению и валидации.

5. LSTM

5.1. Состояние памяти и скрытое состояние

LSTM (long short-term memory) использует отдельное состояние памяти и гейты, управляющие сохранением, записью и чтением информации [1].

В ячейке есть два вектора: состояние памяти c_t и скрытое состояние h_t. Гейты задают покомпонентные коэффициенты сохранения прежней памяти, записи нового содержимого и формирования скрытого состояния.

5.2. Формулы ячейки

Кандидат нового содержимого памяти вычисляется по текущему эмбеддингу и предыдущему скрытому состоянию:

$$g_t = \tanh\big(W_{eg}e_t + W_{hg}h_{t-1} + b_g\big).$$

Долговременная память обновляется так: то, что лежало в ней раньше, плюс текущая информация - но записываемая не всегда, а только если открыт **input gate**:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad i_t = \sigma\big(W_{ei}e_t + W_{hi}h_{t-1} + b_i\big),$$

где $\odot$ - покомпонентное произведение. Сигмоида задает коэффициенты в интервале $(0, 1)$: при значении, близком к единице, соответствующая компонента кандидата записывается в память; при значении, близком к нулю, ее вклад подавляется.

Множитель f_t - это **forget gate**, механизм стирания, устроенный аналогично:

$$f_t = \sigma\big(W_{ef}e_t + W_{hf}h_{t-1} + b_f\big).$$

Без него в память можно было бы только дописывать; forget gate, близкий к нулю, стирает долговременную память, освобождая место под новую информацию.

Скрытое состояние получает информацию из состояния памяти через **output gate**:

$$h_t = o_t \odot \tanh(c_t), \qquad o_t = \sigma\big(W_{eo}e_t + W_{ho}h_{t-1} + b_o\big).$$

Все векторы $g_t, i_t, f_t, o_t, c_t, h_t$ принадлежат $\mathbb{R}^{d_h}$. Матрицы $W_{e*} \in \mathbb{R}^{d_h \times d_e}$ преобразуют входной эмбеддинг, матрицы $W_{h*} \in \mathbb{R}^{d_h \times d_h}$ - предыдущее скрытое состояние, а сдвиги принадлежат $\mathbb{R}^{d_h}$. Функция tanh действует покомпонентно и возвращает значения из интервала $(-1, 1)$. Гиперболический тангенс ограничивает величину сигнала, передаваемого из c_t в h_t. Само состояние памяти c_t при этом не ограничивается интервалом $(-1, 1)$.

На шаге t по e_t, c_{t-1} и h_{t-1} вычисляются три гейта и кандидат g_t, затем состояния c_t и h_t.

5.3. Градиент по состоянию памяти

Основа механизма - аддитивное обновление состояния памяти. Если коэффициент сохранения близок к единице ($f_t \approx 1$), то

$$c_t \approx c_{t-1} + i_t \odot g_t$$

Это аддитивное обновление памяти с прямым путем от c_{t-1} к c_t. На этом прямом пути, при фиксированных значениях гейтов и нового содержимого, производная равна $\mathrm{diag}(f_t)$ и близка к единичной матрице.

Полный градиент учитывает также зависимости гейтов и скрытых состояний. LSTM облегчает передачу информации, но не гарантирует сохранение любого градиента на любой длине.

5.4. Инициализация гейтов

Для гейта забывания используют положительный начальный сдвиг, например $b_f = 1$: это увеличивает начальную долю сохраняемого состояния. Такой эмпирический прием проверялся в экспериментах [4], но не является обязательным условием обучения.

5.5. Вычислительная стоимость

В обычной рекуррентной ячейке используется одно аффинное преобразование входа и предыдущего состояния. В LSTM таких преобразований четыре: для кандидата состояния и трех гейтов. Поэтому при одинаковых d_e и d_h число параметров и объем вычислений примерно в четыре раза больше. Способность хранить информацию на длинных последовательностях зависит от задачи, параметров и режима обучения.

6. GRU

GRU (gated recurrent unit) - гейтовая ячейка с одним состоянием и двумя гейтами [2].

Явного разделения на состояние памяти и скрытое состояние здесь нет - используется один вектор h_t. Два гейта, **reset gate** и **update gate**, вычисляются по формулам

$$r_t = \sigma\big(W_{er}e_t + W_{hr}h_{t-1} + b_r\big), \qquad u_t = \sigma\big(W_{eu}e_t + W_{hu}h_{t-1} + b_u\big).$$

Текущее вычисление - нелинейное преобразование, в котором предыдущая память домножена на reset gate:

$$g_t = \tanh\big(W_{eg}e_t + W_{hg}(r_t \odot h_{t-1}) + b_g\big)$$

При малом r_t соответствующие компоненты предыдущего состояния слабо влияют на кандидат g_t. Обновление состояния контролирует update gate:

$$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot g_t.$$

Если u_t близок к единице, предыдущая информация сохраняется, а текущая игнорируется; близок к нулю - наоборот, записывается текущая. Один update gate управляет сохранением предыдущего состояния и записью нового, связывая эти две доли как u_t и $1 - u_t$. Отдельного output gate в этой схеме нет.

При u_t близком к единице прямой путь сохранения состояния имеет коэффициент, близкий к единице. При одинаковых входной и скрытой размерностях GRU использует три аффинных преобразования вместо четырех у LSTM и обычно содержит меньше параметров.

Векторы r_t, u_t, g_t, h_t принадлежат $\mathbb{R}^{d_h}$. Матрицы W_{e*} имеют размер $d_h \times d_e$, а W_{h*} - размер $d_h \times d_h$.

7. Сборка полной сети

Рекуррентные ячейки объединяют в двунаправленные и многослойные сети. Способ сборки зависит от того, требуется ли один ответ на последовательность, ответ для каждой позиции или выходная последовательность другой длины.

7.1. Двунаправленные сети

В **двунаправленной** (bidirectional) рекуррентной сети на одном уровне работают две сети с разными параметрами. Первая обрабатывает последовательность слева направо, вторая - справа налево. Представление позиции t получают конкатенацией $[\vec{h}_t; \overleftarrow{h}_t]$. Поэтому оно зависит от контекста с обеих сторон. Размер представления и число параметров такого слоя увеличиваются по сравнению с однонаправленным слоем.

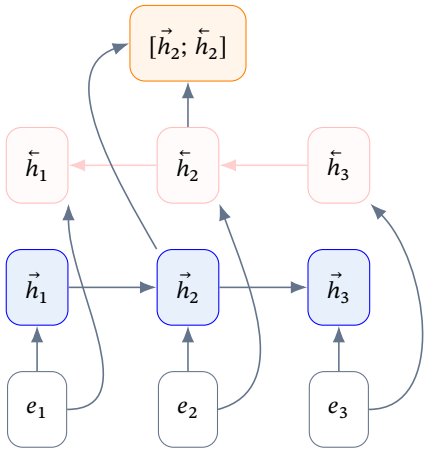


Рис. 3: Двунаправленный слой: одна сеть читает последовательность слева направо, вторая - справа налево; выход для позиции равен конкатенации двух скрытых состояний

Эта конструкция использовалась в ELMo: глубокая двунаправленная языковая модель строит контекстные состояния, а представление токена для прикладной задачи получается из состояний нескольких ее слоев [13]. Формула обучаемой смеси слоев была введена на неделе 6.

7.2. Многослойные сети и понижение временного разрешения

В многослойной сети выходы h_t одного рекуррентного слоя служат входами следующего слоя в той же позиции. Каждый слой имеет собственные параметры и собственную рекуррентную связь.

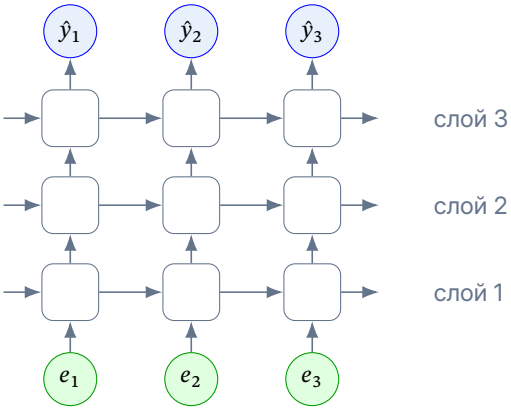


Рис. 4: Многослойная рекуррентная сеть: выход каждого рекуррентного слоя в каждый момент времени служит входом следующего слоя; у каждого слоя своя горизонтальная передача скрытого состояния

Число позиций между рекуррентными слоями можно уменьшать. В пирамидальном кодировщике вход следующего слоя получают конкатенацией соседних состояний [10]:

$$u_k = \mathrm{concat}\left(h^{(\ell)}_{2k-1}, h^{(\ell)}_{2k}\right), \qquad h^{(\ell+1)}_k = \mathrm{RNN}^{(\ell+1)}\left(u_k, h^{(\ell+1)}_{k-1}\right).$$

После такого преобразования число временных позиций сокращается вдвое, а размер вектора увеличивается. Это понижение временного разрешения, а не max-pooling: максимум по компонентам здесь не вычисляется.

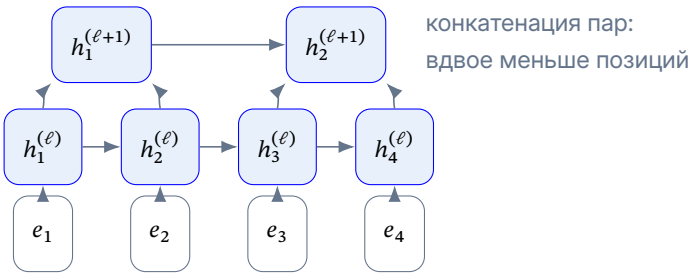


Рис. 5: Понижение временного разрешения между рекуррентными слоями: следующему слою подается конкатенация выходов двух соседних моментов времени

7.3. Остаточные связи по глубине

LSTM и GRU уже имеют прямые пути сохранения состояния внутри ячейки. Дополнительная остаточная связь по времени - отдельный архитектурный выбор. Между слоями глубокой рекуррентной сети также можно добавлять остаточные связи, если размерности позволяют сложение. Они сокращают путь градиента по глубине, но не являются обязательными для каждой рекуррентной модели.

7.4. Архитектуры под типы задач

Классификация последовательности: на входе последовательность, на выходе метка класса. Простейший вариант - прогнать рекуррентную сеть и взять последнее скрытое состояние h_T: это эмбеддинг - представление всей входной последовательности фиксированного размера, - поверх которого ставится полносвязный слой в число классов и кросс-энтропия. Последнее состояние сильнее связано с концом последовательности. Чтобы явно агрегировать состояния всех позиций, можно усреднить h_t по всем моментам времени и подать среднее в полносвязный слой.

Согласованная поэлементная разметка - например, разметка частей речи. Длины входа и выхода совпадают: по состоянию h_t вычисляются логиты классов для позиции t, а позиционные кросс-энтропии суммируются. Другая задача с той же схемой - **распознавание именованных сущностей** (named entity recognition): имен, адресов и дат. Для одного типа однотокенных сущностей достаточно бинарной классификации токенов. Для сущностей из нескольких токенов («ул. Строителей, дом 9») используют BIO-разметку: каждый токен относится к классу B (первый токен сущности), I (продолжение сущности) или O (не сущность), а для нескольких типов сущностей вводят отдельные пары B/I.

Декодировщик: последовательность на выходе имеет заранее неизвестную длину. Генерация начинается с токена `<bos>`. Выбранный токен подается на следующий шаг, и процесс продолжается до токена `<eos>` или ограничения длины. Такая конструкция называется авторегрессионной.

На обучении декодировщик моделирует авторегрессионную вероятность, но получает правильные предыдущие токены из корпуса. Например, вход $x_1, \dots, x_{T-1}$ сопоставляется с целями $x_2, \dots, x_T$: после каждого входного токена предсказывается следующий. Кросс-энтропия суммируется по позициям. Такой режим называется обучением с правильным префиксом (teacher forcing); сама вероятностная модель остается авторегрессионной. Формальное описание для выхода кодировщика-декодировщика приведено ниже.

Если вход - объект фиксированного размера (задача «подпись к картинке»), изображение пропускается через сверточную сеть. Полученное представление либо инициализирует скрытое состояние рекуррентного декодировщика, либо конкатенируется с его входом на каждом шаге. Во втором варианте представление изображения входит в вычисление каждого состояния декодировщика.

Seq2seq: вход и выход имеют произвольные и не обязанные совпадать длины. Кодировщик сворачивает входную последовательность в вектор фиксированного размера. Этот вектор инициализирует декодировщик, который строит выход до токена <eos>.

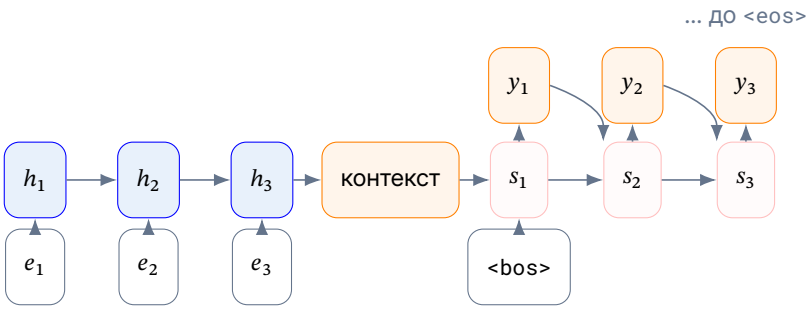


Рис. 6: Кодировщик-декодировщик (seq2seq): вход сворачивается в контекстный вектор фиксированного размера, авторегрессионный декодировщик строит из него выходную последовательность

8. Dropout в рекуррентных сетях

8.1. Маски dropout

При dropout активации покомпонентно умножаются на случайную бинарную маску. При inverted dropout сохраненные компоненты дополнительно делятся на вероятность сохранения, чтобы математическое ожидание не менялось.

Независимые dropout-маски на каждом временном переходе могут затруднять сохранение информации и передачу градиента по длинной цепочке. Один классический вариант применяет dropout к входам и межслойным связям, оставляя рекуррентное состояние без такого обнуления [5]. Это выбор конкретной схемы регуляризации, а не общий запрет на recurrent dropout. Далее рассмотрим варианты, которые специально учитывают временную структуру.

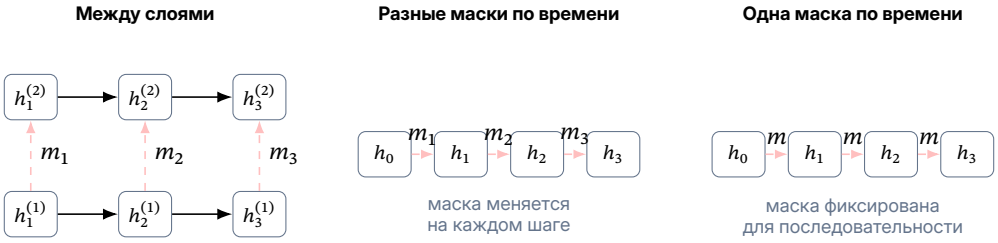


Рис. 7: Три схемы dropout в рекуррентной сети: маска на межслойных связях, разные маски на последовательных временных переходах и одна маска, зафиксированная вдоль последовательности. Последняя схема соответствует вариационному dropout для рекуррентных связей [6].

8.2. Вариационный dropout

В **вариационном dropout** одна маска для данного слоя и объекта используется на всех временных шагах, в том числе в рекуррентных связях [6]. Это сохраняет согласованный набор активных компонент вдоль последовательности и не вносит новый независимый шум на каждом шаге.

8.3. Dropout внутри ячейки

Внутри гейтовой ячейки dropout можно применять к кандидату g_t, который записывается в память, не меняя значения самих гейтов [7]:

$$c_t = f_t \odot c_{t-1} + i_t \odot (m_t \odot g_t).$$

Реализация в PyTorch

Стандартный `nn.LSTM` принимает последовательность и возвращает выходы для всех позиций. Параметр `dropout` применяется между рекуррентными слоями, но не внутри одной ячейки. Для изменения гейтов или рекуррентной маски используют `LSTMCell` и явный цикл по времени.

9. Нормализация в рекуррентных сетях

Скрытое состояние h в развернутой сети имеет три индекса: момент времени, номер последовательности в батче и номер компоненты вектора. Усреднять статистики для нормализации в принципе можно по любому из них.

Батч-нормализация может использоваться в моделях последовательностей, но ее статистики и оси усреднения нужно выбирать с учетом задачи. При небольшом батче оценки могут быть шумными. Layer normalization не требует статистик по другим объектам и потому удобна и при малых, и при больших батчах.

Формулами разница выглядит так. Активации образуют тензор a_{ith} с тремя индексами: i - объект в батче, t - позиция, h - компонента вектора. Батч-нормализация усредняет вдоль батча и времени, статистики свои для каждой компоненты:

$$\mu_h = \frac{1}{BT}\sum_{i=1}^{B}\sum_{t=1}^{T} a_{ith}, \qquad \sigma_h^2 = \frac{1}{BT}\sum_{i=1}^{B}\sum_{t=1}^{T}\big(a_{ith} - \mu_h\big)^2.$$

Layer normalization усредняет вдоль компонент вектора, статистики свои для каждого объекта и каждой позиции [11]:

$$\mu_{it} = \frac{1}{H}\sum_{h=1}^{H} a_{ith}, \qquad \sigma_{it}^2 = \frac{1}{H}\sum_{h=1}^{H}\big(a_{ith} - \mu_{it}\big)^2,$$

дальше в обоих случаях применяются стандартная нормировка и обучаемые масштаб и сдвиг, чтобы тождественное преобразование оставалось достижимым. При последовательностях разной длины статистики батч-нормализации, усредняемые вдоль времени, могут зависеть от позиций заполнения. Layer normalization вычисляет статистики по компонентам одного вектора и не использует другие временные позиции.

Можно также усреднять признаки по времени одного объекта. Но для причинной языковой модели нельзя использовать будущие позиции для нормировки текущего состояния. Формула усреднения по всей

временной оси выше показывает вариант для полностью доступной последовательности; она не задает допустимую причинную нормировку. При применении модели у батч-нормализации обычно используются накопленные статистики, а послойная нормализация вычисляет статистики текущего вектора. Выбор схемы нормализации проверяют в соответствии с протоколом задачи.

10. Обучение авторегрессионного декодировщика

Вернемся к машинному переводу. Вход x дан целиком, а выход $y = (y_1, \dots, y_S)$ генерируется последовательно. Авторегрессионная модель задает

$$p_\theta(y \mid x) = \prod_{t=1}^{S} p_\theta(y_t \mid y_{<t}, x).$$

В выход включают токен конца последовательности <eos>, чтобы модель могла завершить генерацию. На каждом шаге сеть возвращает распределение по словарю выходного языка. Параметр θ обозначает параметры кодировщика, декодировщика и выходного слоя.

Если при обучении сразу использовать только собственные прогнозы, в начале модель будет порождать почти случайный текст. Каждый следующий прогноз будет строиться на ошибочном префиксе. Чтобы обучение получало устойчивый сигнал, применяют teacher forcing: на вход декодировщику подают правильные предыдущие токены $y^*_{<t}$.

$$\mathcal{L}_{\mathrm{CE}}(x, y^*) = -\sum_{t=1}^{S} \log p_\theta(y^*_t \mid y^*_{<t}, x).$$

Это отрицательный логарифм правдоподобия правильного перевода. Известный префикс является частью обучающего примера. Градиент проходит через непрерывные вычисления сети, но не через индексы поданных правильных токенов.

На инференсе правильный префикс неизвестен. Начинаем с <bos>, выбираем первый токен по распределению модели, подаем его обратно и продолжаем до <eos> или ограничения длины. Возникает различие между распределениями префиксов при обучении и при генерации, известное как exposure bias. Ошибка в начале ответа может изменить контекст последующих предсказаний.

Один из предложенных способов учитывать это различие - scheduled sampling [3]. Для предыдущего шага случайно выбирают, подать правильный токен или прогноз модели. Схематически

$$\tilde{y}_{t-1} = \begin{cases} y^*_{t-1}, & \text{с вероятностью } \varepsilon_k, \\ \hat{y}_{t-1}, & \text{с вероятностью } 1 - \varepsilon_k, \end{cases}$$

где k - этап обучения, а ε_k задает долю подсказок. В начале она велика, затем может уменьшаться. Декодировщик видит смешанный префикс и учится предсказывать правильное продолжение.

Эта процедура является эвристикой и меняет обучающий функционал относительно обычного максимального правдоподобия. Она не гарантирует устранения exposure bias и не является обязательной для любой авторегрессионной модели. При дискретном выборе токена обычный градиент не проходит через операцию выбора; эту особенность также требуется учитывать в реализации.

11. Анализ состояний обученной LSTM

Состояние памяти c_t является вектором, компоненты которого меняются вдоль последовательности. В отдельных обученных моделях некоторые компоненты коррелируют с интерпретируемыми признаками: позицией в строке, нахождением внутри кавычек или внутри блока кода. Это наблюдение не означает, что каждой компоненте памяти соответствует устойчивый семантический признак.

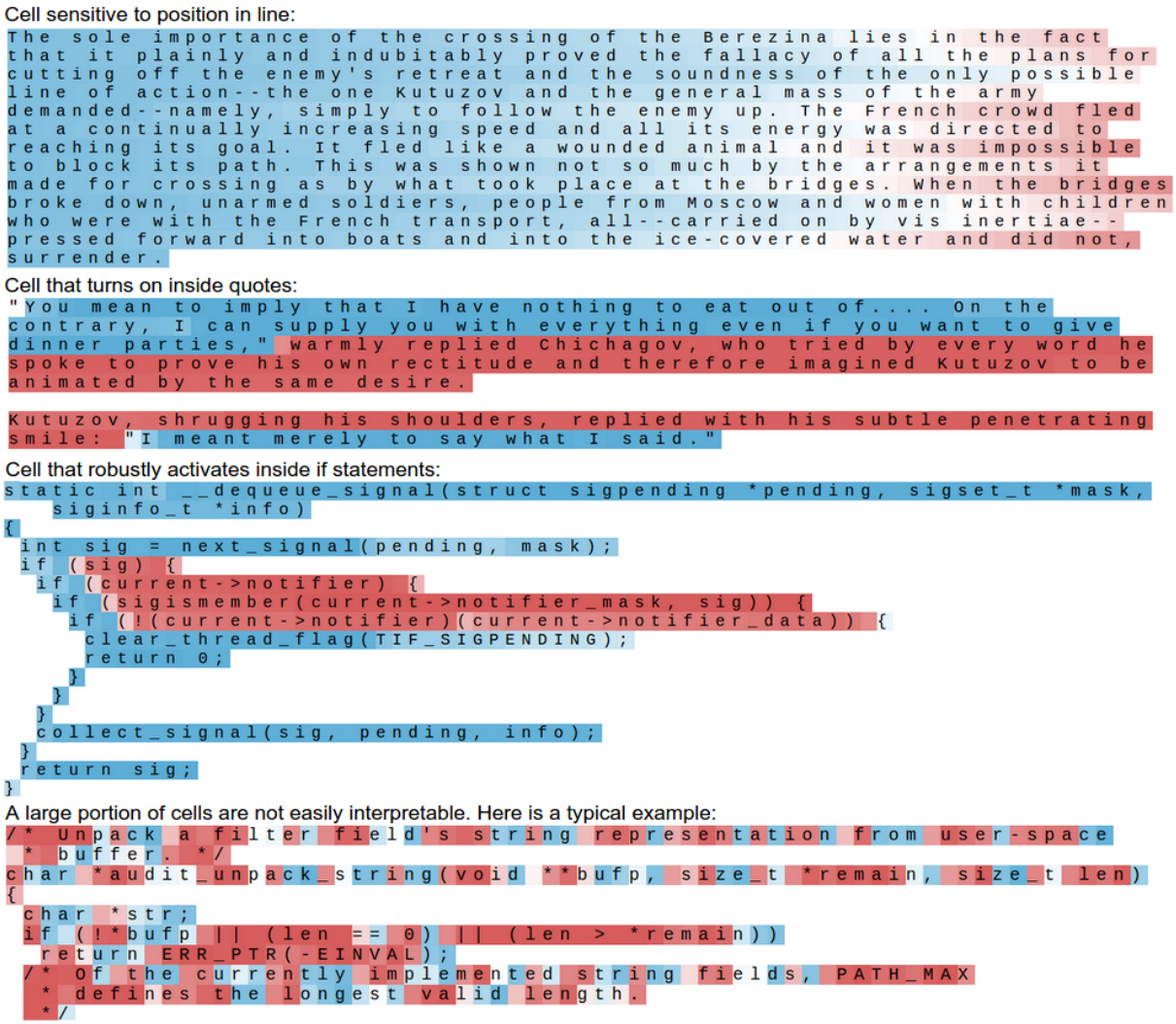


Рис. 8: Активации отдельных компонент состояния памяти посимвольной LSTM. Цвет показывает значение компоненты c_t. Источник: Karpathy et al., 2015, Fig. 2 [12].

12. Ограничение фиксированного контекстного вектора

В базовом кодировщике-декодировщике вся входная последовательность сжимается в один вектор фиксированной размерности. Декодировщик не имеет прямого доступа к промежуточным состояниям кодировщика. При увеличении длины входа этот вектор должен сохранять все больше информации.

Ограничение видно на задаче копирования последовательности в обратном порядке. Кодировщик должен сохранить все элементы входа в последнем состоянии, а декодировщик - восстановить их без прямого доступа к исходным позициям. Механизм внимания снимает это ограничение: на каждом шаге декодировщик строит взвешенную комбинацию состояний кодировщика. Формулы внимания рассматриваются во второй лекции недели 7.

13. Итоги

- Рекуррентная сеть применяет один набор параметров ко всем позициям последовательности и строит контекстные состояния h_t.
- Обратное распространение по времени содержит произведения якобианов. Их нормы определяют затухание и возможный взрыв градиента.
- LSTM и GRU вводят прямые пути сохранения состояния и обучаемые гейты. Они облегчают передачу информации на длинных интервалах, но не дают безусловной гарантии сохранения градиента.
- Двунаправленные и многослойные сети задают представления последовательности, а авторегрессионный декодировщик строит выход между токенами `<bos>` и `<eos>`.
- Базовый кодировщик-декодировщик сжимает вход в один вектор. Механизм внимания заменяет это ограничение доступом к состояниям всех входных позиций.

A. Дополнительные методы стабилизации и регуляризации

A.1. Ортогональная параметризация через матричную экспоненту

Пусть $A \in \mathbb{R}^{d_h \times d_h}$ - обучаемая матрица. Матрица $A - A^\top$ кососимметрична, поэтому

$$W_h = \exp(A - A^\top)$$

ортогональна [9]. Такая параметризация сохраняет ограничение на линейную часть рекуррентного перехода во время обучения. Она остается дифференцируемой, но вычисление матричной экспоненты и ее градиента требует дополнительных матричных операций. Ортогональность W_h не устраняет влияние производной функции активации на полный якобиан.

A.2. Zoneout

В схеме zoneout случайно выбранные компоненты состояния копируются без изменений [8]. Для состояния памяти LSTM обновление можно записать как

$$c_t = m_t \odot c_{t-1} + (1 - m_t) \odot \tilde{c}_t,$$

где $\tilde{c}_t$ - состояние после обычного обновления ячейки, а m_t - бинарная маска. Для компонент с $m_t = 1$ сохраняется прежнее состояние; для остальных используется новое значение.

Список литературы

[1] Hochreiter S., Schmidhuber J. Long Short-Term Memory. Neural Computation, 1997. DOI.

[2] Cho K. et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. 2014. arXiv:1406.1078.

[3] Bengio S. et al. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. 2015. arXiv:1506.03099.

[4] Jozefowicz R., Zaremba W., Sutskever I. An Empirical Exploration of Recurrent Network Architectures. 2015. Статья.

[5] Zaremba W., Sutskever I., Vinyals O. Recurrent Neural Network Regularization. 2014. arXiv:1409.2329.

[6] Gal Y., Ghahramani Z. A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. 2016. arXiv:1512.05287.

[7] Semeniuta S., Severyn A., Barth E. Recurrent Dropout without Memory Loss. 2016. arXiv:1603.05118.

[8] Krueger D. et al. Zoneout: Regularizing RNNs by Randomly Preserving Hidden Activations. 2016. arXiv:1606.01305.

[9] Lezcano-Casado M., Martinez-Rubio D. Cheap Orthogonal Constraints in Neural Networks: A Simple Parametrization of the Orthogonal and Unitary Group. 2019. arXiv:1901.08428.

[10] Chan W. et al. Listen, Attend and Spell: A Neural Network for Large Vocabulary Conversational Speech Recognition. 2016. arXiv:1508.01211.

[11] Ba J. L., Kiros J. R., Hinton G. E. Layer Normalization. 2016. arXiv:1607.06450.

[12] Karpathy A., Johnson J., Fei-Fei L. Visualizing and Understanding Recurrent Networks. 2015. arXiv:1506.02078.

[13] Peters M. E. et al. Deep Contextualized Word Representations. 2018. NAACL 2018.